

Week 3: Seller and Inventory Management System

Scenario:

The e-commerce company sells products supplied by multiple sellers. The system should maintain seller details, track products supplied by each seller, and monitor inventory levels to ensure product availability.

1. Create Seller and Inventory Tables

2. Establish Relationships

Seller and Product

One seller can supply multiple products.

One product can be supplied by one or multiple sellers.

Relationship:

Seller → Product is one to many

Product and Inventory

Each product has inventory information.

Inventory tracks available stock.

Relationship:

Product → Inventory is one to one

```
SQL> CREATE TABLE Seller(Seller_ID VARCHAR(7),Seller_Name VARCHAR(25),Email
VARCHAR(30),Phone NUMBER(10),Address VARCHAR(45));

Table created.

SQL> CREATE TABLE Inventory(Inventory_ID VARCHAR(7),Product_ID VARCHAR(7),Se
ller_ID VARCHAR(7),Stock_Quantity NUMBER,Stock_Status VARCHAR(11),Last_Updat
ed DATE);

Table created.
```

Shows the CREATE TABLE commands used to create the Seller and Inventory tables.

Primary Key added for Both Tables:

```
SQL> ALTER TABLE Seller ADD CONSTRAINT PK_Seller PRIMARY KEY (Seller_ID);

Table altered.

SQL> ALTER TABLE Inventory ADD CONSTRAINT PK_Inventory PRIMARY KEY (Inventory_ID);

Table altered.
```

Shows the primary key constraints added to the Category and Product tables.

Foreign Key for Inventory Table:

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Prod FOREIGN KEY(Product_ID) RE  
FERENCES Product(Product_ID);  
  
Table altered.
```

```
SQL> ALTER TABLE Inventory ADD  
2 CONSTRAINT FK_Seller FOREIGN KEY(Seller_ID)  
3 REFERENCES Seller(Seller_ID);  
  
Table altered.
```

Shows the foreign key constraint added to the Inventory table.

Constraints to make Unique Email ID:

```
SQL> ALTER TABLE Seller ADD CONSTRAINT Un_mail UNIQUE(Email);  
  
Table altered.
```

Shows the unique constraint added to the Seller email column.

To make Email NOT NULL

```
SQL> ALTER TABLE Seller MODIFY Email NOT NULL;  
  
Table altered.
```

Shows the NOT NULL constraint applied to the Seller email column.

Add Constraint to Check the Stock Quantity is Positive

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT CK_Quan CHECK(Stock_Quantity>=0);  
  
Table altered.
```

Shows the CHECK constraint ensuring Stock_Quantity is greater than zero.

Insert Values to Seller Table

```
SQL> INSERT INTO Seller VALUES('&Seller_ID','&Seller_Name','&email','&phone','&Address');
Enter value for seller_id: S10
Enter value for seller_name: Rose Garden Suppliers
Enter value for email: rosegarden@gmail.com
Enter value for phone: 9876500003
Enter value for address: Pallavaram
old 1: INSERT INTO Seller VALUES('&Seller_ID','&Seller_Name','&email','&phone','&Address')
new 1: INSERT INTO Seller VALUES('S10','Rose Garden Suppliers',' rosegarden@gmail.com','9876500003','Pallavaram')

1 row created.

SQL> /
Enter value for seller_id: S20
Enter value for seller_name: Nature Flower Farm
Enter value for email: natureflower@gmail.com
Enter value for phone: 9876500004
Enter value for address: Chromepet
old 1: INSERT INTO Seller VALUES('&Seller_ID','&Seller_Name','&email','&phone','&Address')
new 1: INSERT INTO Seller VALUES('S20','Nature Flower Farm','natureflower@gmail.com','9876500004','Chromepet')

1 row created.

SQL> /
Enter value for seller_id: S30
Enter value for seller_name: Blossom Traders
Enter value for email: blossomtraders@gmail.com
Enter value for phone: 9876500005
Enter value for address: Guindy
old 1: INSERT INTO Seller VALUES('&Seller_ID','&Seller_Name','&email','&phone','&Address')
new 1: INSERT INTO Seller VALUES('S30','Blossom Traders','blossomtraders@gmail.com','9876500005','Guindy')

1 row created.
```

```
SQL> /
Enter value for seller_id: S40
Enter value for seller_name: Fresh Bloom Suppliers
Enter value for email: freshbloom@gmail.com
Enter value for phone: 9876500002
Enter value for address: Tambaram
old 1: INSERT INTO Seller VALUES('&Seller_ID','&Seller_Name','&email','&phone','&Address')
new 1: INSERT INTO Seller VALUES('S40','Fresh Bloom Suppliers','freshbloom@gmail.com','9876500002','Tambaram')

1 row created.

SQL> /
Enter value for seller_id: S50
Enter value for seller_name: Green Garden Flowers
Enter value for email: greengarden@gmail.com
Enter value for phone: 9876500001
Enter value for address: Chennai
old 1: INSERT INTO Seller VALUES('&Seller_ID','&Seller_Name','&email','&phone','&Address')
new 1: INSERT INTO Seller VALUES('S50','Green Garden Flowers','greengarden@gmail.com','9876500001','Chennai')

1 row created.
```

Shows seller records being inserted into the Seller table.

```
SQL> /
Enter value for seller_id: S40
Enter value for seller_name: Fresh Bloom Suppliers
Enter value for email: freshbloom@gmail.com
Enter value for phone: 9876500002
Enter value for address: Tambaram
old 1: INSERT INTO Seller VALUES('&Seller_ID','&Seller_Name','&email','&phone','&Address')
new 1: INSERT INTO Seller VALUES('S40','Fresh Bloom Suppliers','freshbloom@gmail.com','9876500002','Tambaram')

1 row created.

SQL> /
Enter value for seller_id: S50
Enter value for seller_name: Green Garden Flowers
Enter value for email: greengarden@gmail.com
Enter value for phone: 9876500001
Enter value for address: Chennai
old 1: INSERT INTO Seller VALUES('&Seller_ID','&Seller_Name','&email','&phone','&Address')
new 1: INSERT INTO Seller VALUES('S50','Green Garden Flowers','greengarden@gmail.com','9876500001','Chennai')

1 row created.
```

Shows the Seller table after inserting and displaying the seller records.

Report 1: Seller-wise Product Report

Display:

- Seller name
- Number of products supplied
- Available stock

```
SQL> SELECT s.Seller_Name, COUNT(i.Product_ID) AS Number_of_Products, SUM(i.
Stock_Quantity) AS Available_Stock FROM Seller s JOIN Inventory i ON s.Selle
r_ID = i.Seller_ID WHERE i.Stock_Status = 'Available' GROUP BY s.Seller_Name
;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
Nature Flower Farm	1	60
Blossom Traders	1	150

Shows the Seller-wise Product Report with seller name, product count, and available stock.

Use Joins to get details from Seller and Inventory Table.

Insert Values to Inventory Table

```
SQL> INSERT INTO Inventory VALUES('&InvID','&ProdID','&Sell','&Stock','&Stock_status',TO_DATE('&Last_upd','YYYY-MM-DD'));
Enter value for invid: 201
Enter value for prodid: 11
Enter value for sell: S10
Enter value for stock: 100
Enter value for stock_status: Available
Enter value for last_upd: 2026-09-20
old 1: INSERT INTO Inventory VALUES('&InvID','&ProdID','&Sell','&Stock','&Stock_status',TO_DATE('&Last_upd','YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('201','11','S10',100,' Available',TO_DATE(' 2026-09-20','YYYY-MM-DD'))

1 row created.

SQL> /
Enter value for invid: 202
Enter value for prodid: 12
Enter value for sell: S20
Enter value for stock: 60
Enter value for stock_status: Available
Enter value for last_upd: 2026-09-21
old 1: INSERT INTO Inventory VALUES('&InvID','&ProdID','&Sell','&Stock','&Stock_status',TO_DATE('&Last_upd','YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('202','12','S20',60,'Available',TO_DATE(' 2026-09-21','YYYY-MM-DD'))

1 row created.
```

```

SQL> /
Enter value for invId: 203
Enter value for prodId: 13
Enter value for sell: S30
Enter value for stock: 150
Enter value for stock_status: Available
Enter value for last_upd: 2026-09-22
old 1: INSERT INTO Inventory VALUES('&InvID','&ProdID','&Sell','&Stock','&Stock_status',TO_DATE('&Last_upd','YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('203','13','S30',150,'Available',TO_DATE('2026-09-22','YYYY-MM-DD'))

1 row created.

SQL> /
Enter value for invId: 204
Enter value for prodId: 14
Enter value for sell: S40
Enter value for stock: 25
Enter value for stock_status: Low Stock
Enter value for last_upd: 2026-09-23
old 1: INSERT INTO Inventory VALUES('&InvID','&ProdID','&Sell','&Stock','&Stock_status',TO_DATE('&Last_upd','YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('204','14','S40',25,'Low Stock',TO_DATE('2026-09-23','YYYY-MM-DD'))

1 row created.

SQL> /
Enter value for invId: 205
Enter value for prodId: 15
Enter value for sell: S50
Enter value for stock: 105
Enter value for stock_status: Available
Enter value for last_upd: 2026-09-24
old 1: INSERT INTO Inventory VALUES('&InvID','&ProdID','&Sell','&Stock','&Stock_status',TO_DATE('&Last_upd','YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('205','15','S50',105,'Available',TO_DATE('2026-09-24','YYYY-MM-DD'))

1 row created.

```

Shows inventory records being inserted into the Inventory table.

```

SQL> SELECT * FROM Inventory;

INVENTO PRODUCT SELLER_I STOCK_QUANTITY STOCK_STATU LAST_UPDA
-----
201      11      S10                100 Available 20-SEP-26
202      12      S20                60 Available 21-SEP-26
203      13      S30               150 Available 22-SEP-26
204      14      S40                25 Low Stock 23-SEP-26
205      15      S50               105 Available 24-SEP-26

```

Shows the Inventory table records after insertion.

Report 2: Stock Availability Report

Display:

- Product name
- Stock quantity
- Stock status

```

SQL> SELECT p.Product_Name, i.Stock_Quantity, i.Stock_Status FROM Product p
JOIN Inventory i ON p.Product_ID = i.Product_ID;

PRODUCT_NAME
-----
STOCK_QUANTITY STOCK_STATU
-----
Red Rose      100 Available
White Lily    60 Available
Jasmine       150 Available
Purple Orchid 25 Low Stock

```

Shows the Stock Availability Report with product name, stock quantity, and stock status.

Report 3: Inventory Analysis

Find:

- Total products available

```
SQL> SELECT COUNT(*) AS Total_Available_Products FROM Inventory WHERE Stock_
Status = 'Available';

TOTAL_AVAILABLE_PRODUCTS
-----
                        2
```

Shows the total number of products currently available.

- Products out of stock

```
SQL> SELECT COUNT(*) AS OutOfStock_Products FROM Inventory WHERE Stock_Statu
s = 'Out of Stock';

OUTOFSTOCK_PRODUCTS
-----
                        0
```

Shows the number of products that are out of stock.

- Highest stocked products

```
SQL> SELECT p.Product_Name, i.Stock_Quantity FROM Product p JOIN Inventory i
ON p.Product_ID = i.Product_ID WHERE i.Stock_Quantity = (SELECT MAX(Stock_Q
uantity) FROM Inventory);

PRODUCT_NAME
-----
-----
STOCK_QUANTITY
-----
Jasmine
      150
```

Shows the product(s) with the highest stock quantity.

- Average inventory quantity

```
SQL> SELECT AVG(Stock_Quantity) AS Average_Inventory FROM Inventory;

AVERAGE_INVENTORY
-----
                88
```

Shows the average inventory quantity calculated using AVG(Stock_Quantity).